

ROBOT MOBILE AUTONOME

Document Technique

ESP32 • PlatformIO • C++ • FSM

Auteur : Lucas OGER-DOINEAU

Marvin Lorenzo KAMD MEM KAMGANG

Roi que NGOUANA

Formation : Bachelor Robotique & Systèmes Embarqués — YNOV Bordeaux

Date : 02/07/2026

1. Présentation générale du projet

Le Robot Mobile Autonome est un système embarqué à 4 roues conçu pour naviguer de façon autonome dans un environnement inconnu / connu. Il repose sur un microcontrôleur ESP32, deux drivers de moteurs L298N, un capteur ultrasonique monté sur servomoteur pour le balayage de l'environnement, et un capteur PIR pour la détection de présence humaine.

Le comportement du robot est régi par une machine à états finis (FSM) à trois états distincts : déplacement libre (MOVING), évitement d'obstacle (TURNING), et arrêt d'urgence (STOPPED). L'ensemble est développé en C++ avec le framework Arduino sous PlatformIO, dans une architecture multi-fichiers orientée composants.

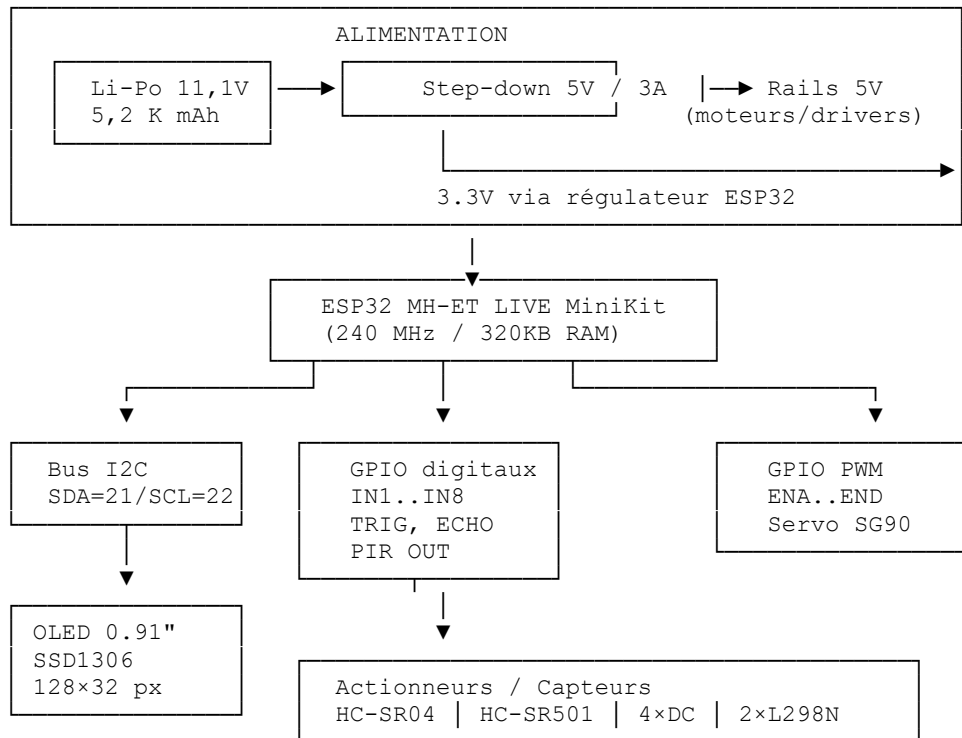
1.1 Objectifs fonctionnels

- Se déplacer librement en avant en l'absence d'obstacle
- Détecter les obstacles via un capteur ultrasonique monté sur servo en balayage
- Effectuer un virage à droite automatique lors de la détection d'un obstacle (< 15 cm)
- S'arrêter immédiatement en cas de détection de mouvement humain (capteur PIR)
- Afficher l'état courant sur un écran OLED embarqué
- Permettre une extension Bluetooth pour le pilotage manuel (perspective future)

2. Architecture matérielle

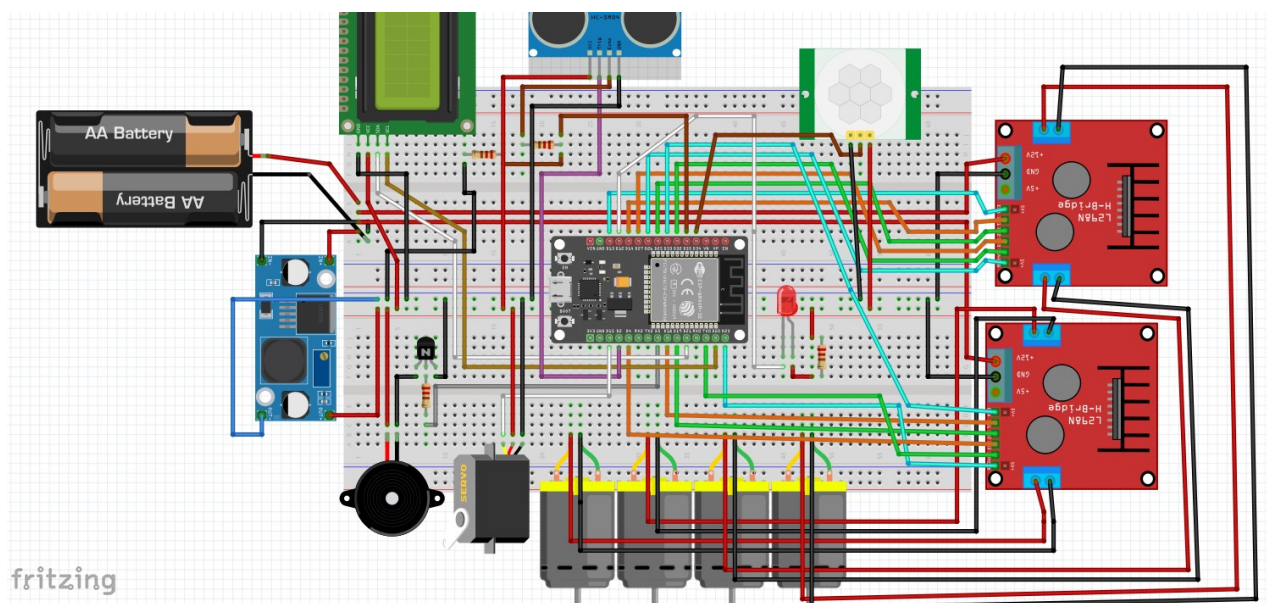
2.1 Schéma bloc général

Le diagramme ci-dessous représente l'architecture matérielle complète du robot, depuis l'alimentation jusqu'aux actionneurs et capteurs :



2.2 Liste des composants

Composant	Référence	Rôle	Qtité
Microcontrôleur	ESP32 MH-ET LIVE MiniKit	Cerveau du robot, WiFi/BT intégré	1
Moteur DC	Moteur DC 3-6V	Propulsion du robot	4
Driver moteur	L298N dual H-Bridge	Pilotage des moteurs DC	2
Capteur ultrason	HC-SR04	Détection d'obstacles (distance)	1
Capteur PIR	HC-SR501	Détection de mouvement infrarouge	1
Servomoteur	SG90	Balayage du capteur ultrason	1
Écran OLED	0.91" 128x32 I2C (SSD1306)	Affichage état du robot	1
Batterie	Li-PO 11,1V / 5,2 K mAh	Alimentation du robot	1
Régulateur	Module step-down 5V	Conversion 12 V→5V pour moteurs/drivers	1



3. Assignment des pins (GPIO mapping)

Toutes les affectations de pins sont centralisées dans le fichier `include/config.h`. Cette approche garantit qu'une modification de câblage se répercute automatiquement sur tout le code sans avoir à chercher dans chaque fichier source.

Composant	Signal	GPIO ESP32	Type	Remarque
SG90 Servo	Signal PWM	GPIO 14	OUTPUT	Fix LEDC : delay(50) après attach()
OLED SSD1306	SDA	GPIO 21	I2C	Bus I2C partagé
OLED SSD1306	SCL	GPIO 22	I2C	Bus I2C partagé
HC-SR04	TRIG	GPIO 12	OUTPUT	Pin strapping — ok après boot
HC-SR04	ECHO	GPIO 35	INPUT ONLY	Pont diviseur
HC-SR501 PIR	OUT	GPIO 34	INPUT ONLY	Calibration 30-60s au boot
L298N n°1 — Moteur A	ENA	GPIO 32	OUTPUT	Enable moteur A
L298N n°1 — Moteur A	IN1	GPIO 33	OUTPUT	Direction A
L298N n°1 — Moteur A	IN2	GPIO 25	OUTPUT	Direction A
L298N n°1 — Moteur B	ENB	GPIO 26	OUTPUT	Enable moteur B
L298N n°1 — Moteur B	IN3	GPIO 27	OUTPUT	Direction B
L298N n°1 — Moteur B	IN4	GPIO 4	OUTPUT	Direction B
L298N n°2 — Moteur C	ENC	GPIO 13	OUTPUT	Enable moteur C
L298N n°2 — Moteur C	IN5	GPIO 15	OUTPUT	Direction C
L298N n°2 — Moteur C	IN6	GPIO 5	OUTPUT	Direction C
L298N n°2 — Moteur D	END	GPIO 18	OUTPUT	Enable moteur D
L298N n°2 — Moteur D	IN7	GPIO 19	OUTPUT	Direction D
L298N n°2 — Moteur D	IN8	GPIO 2	OUTPUT	LED bleue intégrée sur GPIO2

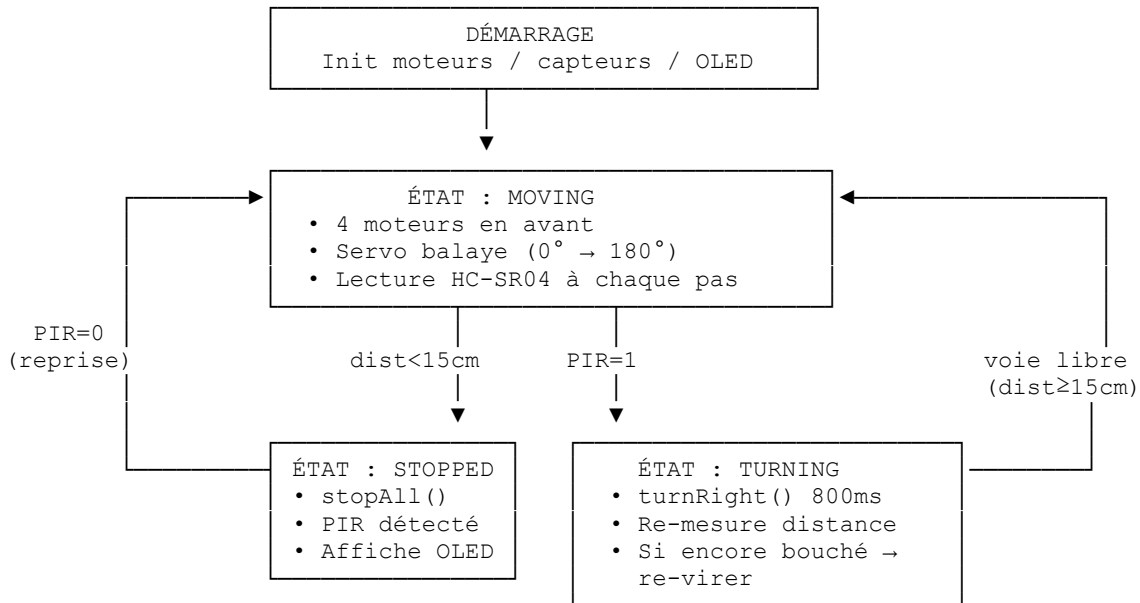
Points d'attention spécifiques

- GPIO35 et GPIO34 sont des pins input-only sur l'ESP32 — ils ne peuvent jamais être configurés en OUTPUT. Ils conviennent parfaitement pour ECHO et PIR qui n'ont besoin que de lire un signal.
- GPIO12 est une pin strapping (MTDI) qui influence la tension flash au boot. Elle peut être utilisée comme GPIO normal une fois le démarrage terminé, mais il faut éviter de la tirer fortement au démarrage.
- GPIO2 est câblé en interne à la LED bleue de la carte MH-ET LIVE. Son utilisation pour IN8 entraîne un clignotement de cette LED synchronisé avec les commandes moteur D — sans impact fonctionnel.
- GPIO1 (TX0) et GPIO3 (RX0) sont réservés à la communication série USB — ne jamais les utiliser pour d'autres fonctions.

4. Analyse fonctionnelle — Machine à états finis (FSM)

4.1 Diagramme des états

Le comportement du robot est décrit par une FSM à 3 états. Le capteur PIR est évalué en priorité absolue à chaque itération de la boucle principale, avant tout autre traitement :



4.2 Description des états

MOVING — Déplacement libre

- Les 4 moteurs DC avancent en simultané
- Le servo SG90 balaye de 0° à 180° par pas de 5°, en rebondissant aux extrémités
- À chaque pas du servo, une mesure de distance est effectuée par le HC-SR04
- Si distance < 15 cm → transition vers TURNING
- Si PIR détecte un mouvement → transition vers STOPPED (prioritaire)

TURNING — Virage à droite

- Les moteurs du côté gauche avancent, les moteurs du côté droit reculent (pivot sur place)
- Le virage dure 800 ms (paramètre TURN_DURATION_MS réglable dans config.h)
- Après le virage, une nouvelle mesure de distance est effectuée
- Si la voie est libre (distance ≥ 15 cm) → retour à MOVING
- Si toujours obstrué → nouveau virage de 800 ms

STOPPED — Arrêt d'urgence

- Tous les moteurs sont stoppés immédiatement (stopAll())
- L'écran OLED affiche 'ARRET / Mouvement PIR!'
- Le robot reste immobile tant que le PIR détecte du mouvement
- Dès que le PIR repasse à LOW → retour automatique à MOVING

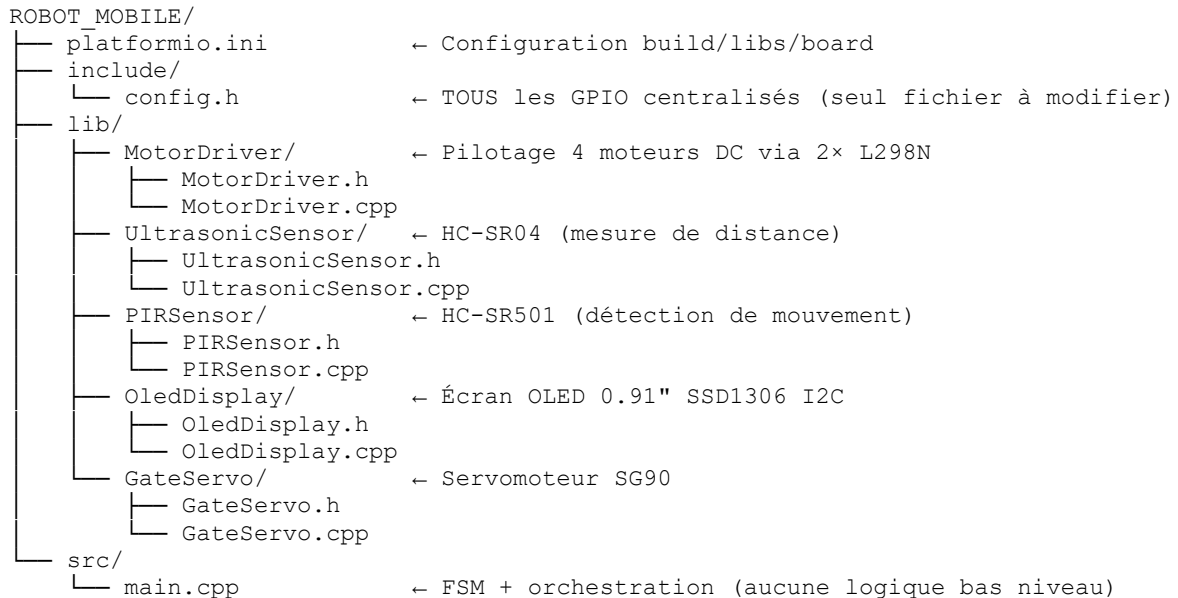
4.3 Paramètres configurables (config.h)

Constante	Valeur défaut	Rôle
OBSTACLE_THRESHOLD_CM	15 cm	Seuil de détection d'obstacle
SERVO_MIN / SERVO_MAX	0° / 180°	Amplitude de balayage du servo
SERVO_STEP	5°	Pas de balayage (finesse de détection)
SERVO_DELAY_MS	30 ms	Délai stabilisation servo avant mesure
TURN_DURATION_MS	800 ms	Durée du virage à droite

5. Architecture logicielle

5.1 Structure des fichiers

Le projet suit une architecture multi-fichiers orientée composants, où chaque module physique correspond à une classe C++ dédiée. Cette approche garantit la lisibilité, la maintenabilité et la réutilisabilité du code.



5.2 Principe de séparation des responsabilités

- Chaque classe expose uniquement des méthodes métier (ex: `motors.forward()`, `oled.showMessage()`) — aucun `digitalWrite()` ne traîne dans `main.cpp`
- `config.h` est le seul point de vérité pour les pins : modifier un GPIO ne nécessite de toucher qu'à ce fichier
- `main.cpp` se lit comme un cahier des charges : les actions sont lisibles en français fonctionnel, pas en détails électroniques
- Les bibliothèques tierces (ESP32Servo, Adafruit SSD1306) sont déclarées uniquement dans `platformio.ini` via `lib_deps` — PlatformIO les télécharge automatiquement

6. Alimentation, consommation et autonomie

6.1 Bilan de consommation

Le tableau suivant présente les consommations typiques et maximales de chaque composant du robot. Les valeurs sont issues des datasheets constructeurs et de mesures empiriques communautaires.

Composant	Tension (V)	Courant typ. (mA)	Courant max (mA)	Source
ESP32	3.3	80	250	3.3V régulateur interne
OLED SSD1306	3.3	20	30	5V step-down
HC-SR04	5.0	15	20	5V step-down
HC-SR501 PIR	5.0	65	65	5V step-down
SG90 (idle)	5.0	10	600	5V step-down
Moteur DC × 4	5.0	1200	4800	L298N
L298N × 2 (logique)	5.0	72	100	5V step-down (2×36mA)
TOTAL ESTIMÉ	—	1462	5865	—

6.2 Calcul de l'autonomie

Hypothèses de calcul

- Batterie : Li-PO → 11,1V / 5200 mAh (configuration minimale)
- Rendement du convertisseur step-down 5V : 85 %
- Profil d'utilisation : 70% MOVING (moteurs à charge partielle), 10% TURNING, 20% STOPPED

Courant moyen selon profil :

$$I_{\text{moy}} = (0.70 \times 1200) + (0.10 \times 800) + (0.20 \times 262) = 840 + 80 + 52 = 972 \text{ mA}$$

Courant réel tiré de la batterie (prise en compte rendement step-up) :

$$I_{\text{batterie}} = (I_{\text{moy}} \times 5V) / (7.4V \times 0.85) = (972 \times 5) / 6.29 \approx 773 \text{ mA}$$

Autonomie estimée :

$$T = \text{Capacité} / I_{\text{batterie}} = 5200 \text{ mAh} / 773 \text{ mA} \approx 6\text{h}45 \text{ min environ.}$$

7. Perspective : Contrôle via Bluetooth (BLE/BT Classic)

7.1 Motivation et intérêt

L'ESP32 intègre nativement un module Bluetooth 4.2 (BT Classic + BLE) ainsi que le WiFi 802.11 b/g/n, sans nécessiter de composant externe. Cette capacité constitue une extension naturelle du projet pour permettre un pilotage manuel ou semi-automatique du robot depuis un smartphone ou une application dédiée.

L'intégration Bluetooth ne modifie pas le câblage existant — aucune pin supplémentaire n'est requise.

7.2 Modes de contrôle envisagés

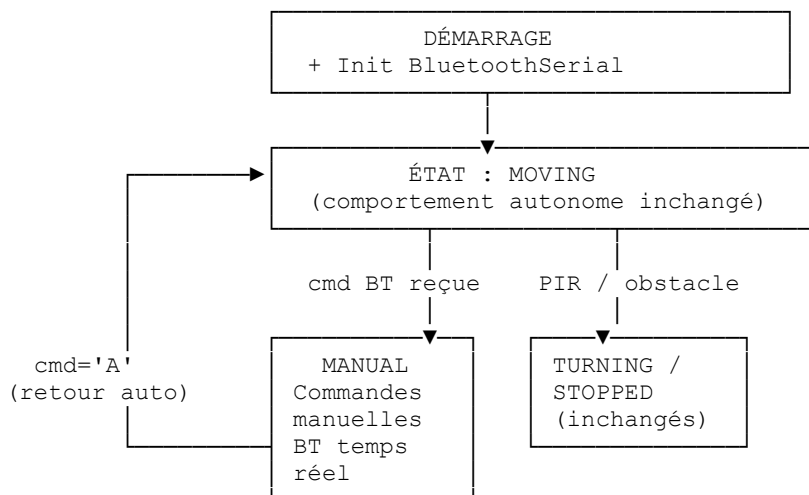
Mode Bluetooth Classic (SPP — Serial Port Profile)

- Émulation d'un port série sans fil via la lib `BluetoothSerial.h` (incluse dans le framework Arduino ESP32)
- Compatible avec les applications Android/iOS de type 'Serial Bluetooth Terminal'
- Commandes textuelles simples : F (forward), B (backward), R (right), L (left), S (stop), A (auto)
- Facilement intégrable dans l'architecture FSM existante via un nouvel état MANUAL

Mode BLE (Bluetooth Low Energy)

- Consommation réduite par rapport au BT Classic — adapté à une utilisation sur batterie
- Utilisation de la lib `NimBLE-Arduino` pour une empreinte mémoire minimale
- Permet la communication avec des applications Flutter/React Native pour une interface personnalisée

7.3 Architecture FSM étendue avec Bluetooth



7.4 Extrait de code d'intégration prévu

```
// Dans main.cpp — ajout minimal pour BT Classic
#include <BluetoothSerial.h>
BluetoothSerial bt;

void setup() {
  bt.begin("RobotMobile"); // nom visible au scan BT
  // ... reste de l'init ...
}

void loop() {
```

```
// Priorité BT si un opérateur envoie une commande
if (bt.available()) {
    char cmd = bt.read();
    state = RobotState::MANUAL;
    handleManualCmd(cmd); // F/B/R/L/S/A
}
// ... reste de la FSM ...
}
```

7.5 Contraintes et points d'attention

- BluetoothSerial et WiFi ne peuvent pas être actifs simultanément sur l'ESP32 (partage d'antenne RF) — pour une utilisation conjointe WiFi+BT, il faut utiliser BLE uniquement
- Le Bluetooth Classic consomme ~50-100 mA supplémentaires en transmission active — à intégrer dans le bilan d'autonomie
- La portée typique est de 10-30 mètres en champ libre selon les obstacles et les appareils
- Aucune pin GPIO supplémentaire n'est nécessaire — l'intégration est 100% logicielle

8. Conclusion et perspectives

Le Robot Mobile Autonome constitue une plateforme embarquée complète intégrant acquisition sensorielle, traitement temps réel, actionnement et interface homme-machine locale (OLED). L'architecture logicielle modulaire adoptée (FSM + classes par composant) permet une évolution aisée du projet sans remise en cause de l'existant.

Les perspectives d'évolution identifiées à court et moyen terme sont les suivantes :

- Intégration du contrôle Bluetooth Classic pour le pilotage manuel via smartphone
- Ajout d'un algorithme de suivi de ligne ou de cartographie simple (SLAM basique)
- Remplacement des drivers L298N par des drivers plus efficaces (DRV8833) pour améliorer l'autonomie et réduire les pertes thermiques
- Ajout d'une caméra OV2640 compatible ESP32 pour la surveillance vidéo en temps réel via WiFi
- Migration vers un PID sur la vitesse des moteurs si des encodeurs sont ajoutés aux roues